@ConfigurationProperties

Lets first see what's the issue with @Value annotation

Its good for injecting a single property, but it becomes problematic as configuration grows.

```
@Component
public class DBConnection {

    @Value("${app.username}")
    private String username;

    @Value("${app.password}")
    private String password;

    @Value("${app.db.url}")
    private String dbUrl;
}
```

- Too much duplication of @Value annotation.
- No way to validate:

  for example:
- password must be present.
- password length 10<=password<=25

Concept && Coding

@ConfigurationProperties:

- It maps configuration (from application.properties) into Java Object.
- Makes configuration structured, reusable and validated.

application.properties:

```
user.name=test_username
user.age=27
user.active=true

```

Binds to:

```
@Component
@ConfigurationProperties(prefix = "user")
public class UserConfigurations {

    private String name;
    private int age;
    private boolean active;

    // getters and setters
}

```

We can simply inject the UserConfiguration and access the properties just like java objects:

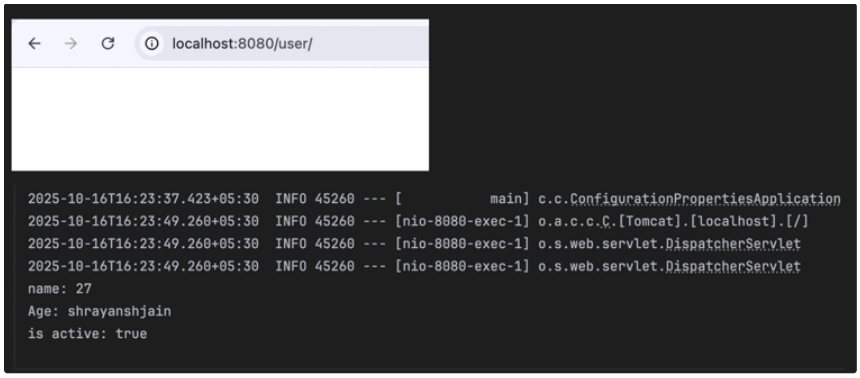
```
@RestController
```

```java
@RequestMapping("/user")
public class UserController {

    @Autowired
    UserConfigurations userConfigurations;

    @GetMapping("/")
    public void getUserConfig() {
        System.out.println("name: " + userConfigurations.getAge());
        System.out.println("Age: " + userConfigurations.getName());
        System.out.println("is active: " + userConfigurations.isActive());
    }
}

```



Handling nested/complex configuration:

Concept && Coding

`application.properties`

```
user.name=test_username
user.age=27
user.active=true
user.address.city=myCityName
user.address.country=myCountryName

```

`UserConfigurations.java`

```java
@Component
@ConfigurationProperties(prefix = "user")
public class UserConfigurations {

    private String name;
    private int age;
    private boolean active;
    private AddressConfig address;

    /*
     Why static? bcoz
    • Spring creates UserConfigurations bean (because it is @Component).

    • It sees a field AddressConfig address.

    • Binder uses reflection and tries to instantiate the type
       AddressConfig using the default no-arg constructor.

```

```
    • If the class is non-static, there is no default no-arg
      constructor. By default it add one constructor which take
       outer class parameter like InnerClass(OuterClass obj) { }

      And since, binder does not find any no-arg constructor,
      Reflection fails → binding fails →address field stays null
      That's the reason, we have to create nested class static
      for automatic binding.
    */

    public static class AddressConfig {
        private String city;
        private String country;

        // getters and setters
    }

    // getters and setters
}

```

Configurations as List:

`application.properties`

```
user.name=test_username
user.age=27
user.active=true
user.address.city=myCityName
user.address.country=myCountryName

#List<String>
user.roles[0]=ADMIN
user.roles[1]=EDITOR

#List<Object>
user.course[0].name=Java
user.course[0].enrolled=true

user.course[1].name=Springboot
user.course[1].enrolled=false

```

Concept && Coding

`UserConfigurations.java`

```
@Component
@ConfigurationProperties(prefix = "user")
public class UserConfigurations {

    private String name;
    private int age;
    private boolean active;
    private AddressConfig address;
    private List<String> roles;
    private List<Course> course;

    public static class AddressConfig {
        private String city;
        private String country;

        // getter and setter
    }

    public static class Course {

```

```java
        String name;
        boolean enrolled;

        // getter and setter
    }

    // getters and setters
}

```

`UserController.java`

```java
@RestController
@RequestMapping("/user")
public class UserController {

    @Autowired
    UserConfigurations userConfigurations;

    @GetMapping("/")
    public void getUserConfig() {
        System.out.println("name: " + userConfigurations.getAge());
        System.out.println("Age: " + userConfigurations.getName());
        System.out.println("is active: " + userConfigurations.isActive());
        System.out.println("Address: " +
                userConfigurations.getAddress().getCity() + ":" +
                userConfigurations.getAddress().getCountry());
        System.out.println("Roles: " +
                userConfigurations.getRoles().get(0) + ":" +
                userConfigurations.getRoles().get(1));
        System.out.println("Courses: " +
                userConfigurations.getCourse().get(0).getName() + ":" +
                userConfigurations.getCourse().get(1).getName());
    }
}

```

Concept && Coding

```
2025-10-16T22:24:06.216+05:30  INFO 47206 --- [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]
2025-10-16T22:24:06.216+05:30  INFO 47206 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
2025-10-16T22:24:06.217+05:30  INFO 47206 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
name: 27
Age: shrayanshjain
is active: true
Address: myCityName:myCountryName
Roles: ADMIN:EDITOR
Courses: Java:Springboot
```

Configurations as Map:

`application.properties`

```properties
user.name=test_username
user.age=27
user.active=true
user.address.city=myCityName
user.address.country=myCountryName

#List<String>
user.roles[0]=ADMIN
user.roles[1]=EDITOR

```

```
#List<Object>
user.course[0].name=Java
user.course[0].enrolled=true

user.course[1].name=Springboot
user.course[1].enrolled=false

#Can be used as nested class and Map too.
user.preferences.theme=dark
user.preferences.language=en
user.preferences.timezone=IST

```

`UserConfigurations.java`

```java
@Component
@ConfigurationProperties(prefix = "user")
public class UserConfigurations {

    private String name;
    private int age;
    private boolean active;
    private AddressConfig address;
    private List<String> roles;
    private List<Course> course;
    Map<String, String> preferences; //configuration as Map

    public static class AddressConfig {
        private String city;
        private String country;

        // getter and setter
    }

    public static class Course {

        String name;
        boolean enrolled;

        // getter and setter
    }

    // getters and setters
}

```

Concept && Coding

`UserController.java`

```java
@RestController
@RequestMapping("/user")
public class UserController {

    @Autowired
    UserConfigurations userConfigurations;

    @GetMapping("/")
    public void getUserConfig() {
        System.out.println("name: " + userConfigurations.getAge());
        System.out.println("Age: " + userConfigurations.getName());
        System.out.println("is active: " + userConfigurations.isActive());
        System.out.println("Address: " +
                userConfigurations.getAddress().getCity() + ":" +
                userConfigurations.getAddress().getCountry());
        System.out.println("Roles: " +
                userConfigurations.getRoles().get(0) + ":" +
```

```
                userConfigurations.getRoles().get(1));
        System.out.println("Courses: " +
                userConfigurations.getCourse().get(0).getName() + ":" +
                userConfigurations.getCourse().get(1).getName());

        //configuration as Map
        System.out.println("Preferences: " +
                userConfigurations.getPreferences().get("theme"));
    }
}

```

Map of an Object:

`application.properties`

```
user.name=test_username
user.age=27
user.active=true
user.address.city=myCityName
user.address.country=myCountryName

#List<String>
user.roles[0]=ADMIN
user.roles[1]=EDITOR

#List<Object>
user.course[0].name=Java
user.course[0].enrolled=true

user.course[1].name=Springboot
user.course[1].enrolled=false

#Map<String, String> preferences
user.preferences.theme=dark
user.preferences.language=en
user.preferences.timezone=IST

#Map<String, Address>
#home -> {city=Jaipur, state=Rajasthan}
#office->{city=Delhi, state=Delhi}
user.locations.home.city=Jaipur
user.locations.home.country=India

user.locations.office.city=Delhi
user.locations.office.country=India

```

Concept && Coding

`UserConfigurations.java`

```
@Component
@ConfigurationProperties(prefix = "user")
public class UserConfigurations {

    private String name;
    private int age;
    private boolean active;
    private AddressConfig address;
    private List<String> roles;
    private List<Course> course;
    Map<String, String> preferences;
    Map<String, AddressConfig> locations; //map of an object

    public static class AddressConfig {
```

```java
        private String city;
        private String country;

        // getters and setters
    }

    public static class Course {

        String name;
        boolean enrolled;

        // getters and setters
    }

    // getters and setters
}

```

`UserController.java`

```java
@RestController
@RequestMapping("/user")
public class UserController {

    @Autowired
    UserConfigurations userConfigurations;

    @GetMapping("/")
    public void getUserConfig() {
        System.out.println("name: " + userConfigurations.getAge());
        System.out.println("Age: " + userConfigurations.getName());
        System.out.println("is active: " + userConfigurations.isActive());
        System.out.println("Address: " +
                userConfigurations.getAddress().getCity() + ":" +
                userConfigurations.getAddress().getCountry());
        System.out.println("Roles: " +
                userConfigurations.getRoles().get(0) + ":" +
                userConfigurations.getRoles().get(1));
        System.out.println("Courses: " +
                userConfigurations.getCourse().get(0).getName() + ":" +
                userConfigurations.getCourse().get(1).getName());
        System.out.println("Preferences: " +
                userConfigurations.getPreferences().get("theme"));

        //map of an object
        System.out.println("Location: " +
userConfigurations.getLocations().get("home").getCity());
    }
}

```

Concept && Coding

```
2025-10-16T23:00:04.935+05:30  INFO 47658 --- [           main] c.c.ConfigurationPropertiesApplication
2025-10-16T23:00:08.068+05:30  INFO 47658 --- [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]
2025-10-16T23:00:08.068+05:30  INFO 47658 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
2025-10-16T23:00:08.069+05:30  INFO 47658 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
name: 27
Age: shrayanshjain
is active: true
Address: myCityName:myCountryName
Roles: ADMIN:EDITOR
Courses: Java:Springboot
Preferences: dark
Location: Jaipur
```

Validations:

Spring Boot automatically validates fields **after binding**, before the app fully starts.

We need to add below dependency:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

```

- We need to add @Validated on your @ConfigurationProperties class.
- If any validation fails, application will fail to **startup.**

`UserConfigurations.java`

```
@Component
@ConfigurationProperties(prefix = "user")
@Validated
public class UserConfigurations {

    @NotBlank(message = "name must not be empty")
    private String name;

    @Min(value = 1, message = "Age can not be 0")
    private int age;

    // getters and setters
}

```

Concept && Coding

`application.properties`

```
user.name=test_username
user.age=0
user.active=true
user.address.city=myCityName
user.address.country=myCountryName

```

```
2025-10-17T17:08:39.381+05:30  INFO 49411 --- [           main] o.apache.catalina.core.StandardService   : Stopping service [Tomcat]
2025-10-17T17:08:39.385+05:30  INFO 49411 --- [           main] .s.b.a.l.ConditionEvaluationReportLogger :

Error starting ApplicationContext. To display the condition evaluation report re-run your application with 'debug' enabled.
2025-10-17T17:08:39.390+05:30 ERROR 49411 --- [           main] o.s.b.d.LoggingFailureAnalysisReporter   :

***************************
APPLICATION FAILED TO START
***************************

Description:

Binding to target com.concepts.UserConfigurations failed:

    Property: user.age
    Value: "0"
    Origin: class path resource [application.properties] - 2:10
    Reason: Age can not be 0
```

Annotation	Description	Example

@NotNull	Value cannot be null	@NotNull String name;
@NotBlank	Not null or empty (for String)	@NotBlank String username;
@NotEmpty	Not null or empty (for List, Map, etc.)	@NotEmpty List<String> courses;
@Min(value)	Minimum value (for numeric types)	@Min(1) int age;
@Max(value)	Maximum value	@Max(100) int age;
@Positive / @PositiveOrZero	Must be > 0 / >= 0	@Positive int retryCount;
@Negative / @NegativeOrZero	Must be < 0 / <= 0	@Negative int loss;
@Email	Must be valid email format	@Email String email;
@Pattern(regexp)	Must match regex	@Pattern(regexp = "[A-Za-z]*") String userName;
@AssertTrue / @AssertFalse	Must be true/false	@AssertTrue boolean isActive;

Concept && Coding

Now, one question comes to our mind is:

Above automatic binding is using setter method, which means it can not be immutable, so how can we make @ConfigurationProperties class immutable.

Answer is: **Constructor Binding**

`ImmutableUserConfiguration.java`

```java
@ConfigurationProperties(prefix = "user")
public class ImmutableUserConfiguration {

    private final String name;
    private final int age;
    private final boolean active;

    ImmutableUserConfiguration(String name, int age, boolean active) {

        this.name = name;
        this.age = age;
        this.active = active;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public boolean isActive() {
        return active;
    }
}

```

Concept && Coding

`ConfigurationPropertiesApplication.java`

```java
@SpringBootApplication
@ConfigurationPropertiesScan
public class ConfigurationPropertiesApplication {

    public static void main(String[] args) {
        SpringApplication.run(ConfigurationPropertiesApplication.class, args);
    }
}

```

Notice that in above 2 classes, *we have not provided @Component, because, we don't have any default constructor and also spring IOC don't know what value to pass in the constructor parameter.*

So that's, why we are giving the bean creation responsibility to Configuration Binder. It invoke the constructor with proper property values, as there is no setter method in immutable class.

Earlier its not an issue, because we have default constructor, so Spring IOC able to create the empty bean and later configuration binder can use setter methods to update those fields.

`UserController.java`

```java
@RestController
@RequestMapping("/user")
public class UserController {
```

```java

    @Autowired
    ImmutableUserConfiguration immutableUserConfiguration;

    @GetMapping("/")
    public void getUserConfig() {

        //immutable
        System.out.println("immutable name: " + immutableUserConfiguration.getName());
        System.out.println("immutable Age: " + immutableUserConfiguration.getAge());
        System.out.println("immutable is active: " + immutableUserConfiguration.isActive());
    }
}

```

```
immutable name: shrayanshjain
immutable Age: 27
immutable is active: true
```

There is no difference in validation of the fields.

`ImmutableUserConfiguration.java`

```java
@ConfigurationProperties(prefix = "user")
@Validated
public class ImmutableUserConfiguration {

    private final String name;

    @Min(1)
    private final int age;

    private final boolean active;

    ImmutableUserConfiguration(String name, int age, boolean active) {

        this.name = name;
        this.age = age;
        this.active = active;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public boolean isActive() {
        return active;
    }
}

```

Concept && Coding